

Servidores web Apache y Nginx

Instalación y configuración

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI · Servicios de Red e Internet

SRI · UNIDAD 2 — SERVIDORES WEB: APACHE Y NGINX

01

Servidores web

Conceptos comunes a Apache y Nginx

¿Qué es un servidor web?

- Programa que **implementa el protocolo HTTP**
- Sirve **páginas estáticas** (HTML, CSS, JS, imágenes...)
- Sirve **páginas dinámicas** generadas por lenguajes como **PHP, Python, Java...**
- Suele apoyarse en un **servidor de aplicaciones** para ejecutar el código
- Implementa funcionalidades del protocolo: **virtual hosts, redirecciones, autenticación, control de acceso...**

Virtual hosts

Un **virtual host** permite que un mismo servidor atienda **varios sitios web** desde una **única dirección IP**.

- Cada sitio tiene su propio **contenido, configuración y logs**
- El servidor elige el sitio por la cabecera **Host** de la petición (*virtual hosts basados en nombre*)
- Si el **Host** no coincide con ningún sitio (por ejemplo, al entrar por la IP), responde el **sitio por defecto**



Sin la cabecera **Host** el servidor no sabría qué sitio servir: por eso es **obligatoria** en HTTP/1.1.

De la URL al fichero

Cada sitio tiene un **directorio raíz** (`DocumentRoot` en Apache, `root` en Nginx): la ruta de la URL se busca a partir de él. Con `www.sitio1.org` → `/var/www/sitio1` y `www.sitio2.org` → `/var/www/sitio2`:

URL PEDIDA	FICHERO	RESPUESTA
<code>http://www.sitio1.org/docs/a.html</code>	<code>/var/www/sitio1/docs/a.html</code>	200 con el fichero
<code>http://www.sitio2.org/docs/a.html</code>	<code>/var/www/sitio2/docs/a.html</code>	Mismo servidor, otro sitio
<code>http://www.sitio1.org/</code>	<code>/var/www/sitio1/index.html</code>	200 con el fichero índice
<code>http://www.sitio1.org/docs/</code>	<code>/var/www/sitio1/docs/index.html</code>	Si no hay índice: listado (si se permite) o 403
<code>http://www.sitio1.org/docs</code>	Es un directorio	301 a <code>/docs/</code>
<code>http://www.sitio1.org/info.php</code>	<code>/var/www/sitio1/info.php</code>	Se ejecuta y se devuelve el HTML generado
<code>http://www.sitio1.org/nada.html</code>	No existe	404

El usuario `www-data`

En Debian, Apache y Nginx atienden las peticiones con el usuario sin privilegios `www-data`:

- El proceso principal arranca como `root` (hace falta para abrir el puerto 80) y los procesos que atienden las peticiones son de `www-data`
- Si alguien aprovecha un fallo de la web, actúa como `www-data`, **no como** `root`
- Necesita **lectura** (`r`) en los ficheros y **paso** (`x`) en **todos** los directorios de la ruta; escritura solo si la aplicación guarda algo

```
ps aux | grep -E 'apache2|nginx'      # usuario de cada proceso
namei -l /home/usuario/doc/a.pdf     # permisos de cada directorio de la ruta
```

 En Debian los directorios personales se crean con permisos `700`: lo que esté en `/home/usuario` da **403** hasta que `www-data` pueda atravesarlo.

Cómo probar los virtual hosts

Sin DNS, el cliente necesita **resolver los nombres** del sitio a la IP del servidor:

```
# /etc/hosts del cliente
10.0.0.1    www.sitio1.org www.sitio2.org
```

Con `curl` también se puede **indicar el Host** sin tocar la resolución:

```
curl http://www.sitio1.org           # resuelve con /etc/hosts
curl -H "Host: www.sitio2.org" http://10.0.0.1 # mismo servidor, otro sitio
curl http://10.0.0.1                 # sitio por defecto
```

Alias

Un **alias** asocia una URL a una **ruta del sistema de ficheros** distinta del directorio raíz del sitio, sin mover los ficheros.

`http://servidor/imagenes/logo.png` → `/srv/recursos/img/logo.png`

- Útil para publicar directorios que están **fuera del directorio raíz**
- El usuario del servidor web (`www-data`) necesita **permisos** sobre la ruta real

Redirección y reescritura

REDIRECCIÓN

- El servidor responde con un **3xx** y la cabecera `Location`
- El cliente hace una **nueva petición** a esa URL
- La URL **cambia** en el navegador
- **301** permanente · **302** temporal

REESCRITURA

- El servidor cambia la ruta **internamente**
- Responde directamente con el recurso (**200**)
- La URL **no cambia** en el navegador
- Sirve para **URL amigables**: `/producto/5` → `/producto.php?id=5`

Autenticación básica

Mecanismo para que el cliente se identifique con **usuario y contraseña** antes de obtener un recurso.

- Sin credenciales, el servidor responde **401** y el navegador pide usuario y contraseña
- El navegador las envía **en cada petición**, codificadas en **Base64**:

```
Authorization: Basic dXNlcjpwYXNz
$ echo dXNlcjpwYXNz | base64 -d
user:pass
```

- Existe también la autenticación *Digest*, que envía un *hash*, pero apenas se usa

 Usa **siempre HTTPS** con autenticación: Base64 es una **codificación**, no un cifrado. Cualquiera que capture el tráfico obtiene la contraseña. HTTPS cifra la conexión completa, cabeceras incluidas.

Control de acceso

Restringe quién puede acceder a determinados recursos o directorios. Puede combinar varios criterios:

- **Dirección IP** o red de origen
- **Nombre de usuario y contraseña**
- **Nombres de dominio** del cliente

Si se deniega el acceso, el servidor responde **403 Forbidden**.



Los criterios se pueden **combinar**: que se cumplan todos o que baste con uno (por ejemplo, entrar sin contraseña desde la red interna y con contraseña desde fuera).

Logs

Los dos servidores guardan dos registros principales:

- `access.log` — una línea por **petición** atendida
- `error.log` — errores de configuración, permisos, backends...

```
10.0.0.2 - pepe [06/Oct/2026:10:15:32 +0200] "GET /secreto/ HTTP/1.1" 200 512 "-" "curl/8.14.1"
```

IP del cliente · usuario autenticado · fecha · **línea de petición** · **código de estado** · tamaño · *referer* (página desde la que se llegó) · *user agent* (cliente)

 Ante un error, mira el **código de estado** en el `access.log` y el motivo en el `error.log`.

SRI · UNIDAD 2 — SERVIDORES WEB: APACHE Y NGINX

02

Apache2

Instalación, configuración y funcionalidades

Apache HTTP Server

- Servidor web **HTTP de código abierto**, multiplataforma
- Desarrollado por la **Apache Software Foundation** (proyecto *httpd*)
- Surgió en 1995 a partir del servidor **NCSA HTTPd**
- **Servidor web más usado en Internet entre 1996 y 2020**
- Destaca por su **estabilidad, modularidad** y comunidad
- Integración sencilla con **PHP, Python, Perl, CGI...**
- Atiende las conexiones con **procesos e hilos**, según el **MPM** que se use
- Versión estable actual: **2.4**

Módulos de multiprocesamiento (MPM)

El **MPM** decide cómo reparte Apache las conexiones entre procesos e hilos. Solo puede haber **uno** activo.

MPM	CÓMO FUNCIONA	CUÁNDO SE USA
prefork	Un proceso por conexión, sin hilos	Con módulos que no admiten hilos, como <code>mod_php</code> . Consume más memoria
worker	Varios procesos, cada uno con varios hilos ; un hilo por conexión	Menos memoria que prefork
event	Como worker, pero las conexiones <i>keep-alive</i> en espera no ocupan un hilo	Por defecto en Debian. El más eficiente

```
apache2ctl -V | grep MPM                # MPM activo
sudo a2dismod mpm_event && sudo a2enmod mpm_prefork
```

Instalación de Apache

```
sudo apt update  
sudo apt install apache2
```

- El servicio queda **levantado y habilitado** (`apache2.service`)
- Escucha por defecto en el puerto **80**
- Sirve contenido desde `/var/www/html/`

```
systemctl reload apache2    # recargar configuración sin cortar conexiones  
systemctl restart apache2   # reiniciar el servicio
```

Ejercicio rápido 1: Instalar Apache

≈ 5 MINUTOS

En una máquina virtual o una instancia de OpenStack (el **servidor**):

1. Instala `apache2` y comprueba que el servicio está activo.
2. Desde tu equipo: `curl -I http://<ip de acceso>`. ¿Qué dice la cabecera `Server`?
3. `ps aux | grep apache2`: ¿qué usuario tiene cada proceso? ¿Por qué?
4. ¿Qué MPM está usando? (`apache2ctl -V`)

Estructura de `/etc/apache2/`

RUTA	PARA QUÉ SIRVE
<code>apache2.conf</code>	Archivo principal de configuración global
<code>ports.conf</code>	Puertos en los que escucha el servidor
<code>sites-available/</code> · <code>sites-enabled/</code>	Virtual hosts disponibles · activos
<code>mods-available/</code> · <code>mods-enabled/</code>	Módulos disponibles · cargados
<code>conf-available/</code> · <code>conf-enabled/</code>	Fragmentos de configuración disponibles · activos

i Los directorios `*-enabled` contienen **enlaces simbólicos** a los `*-available`: la configuración se prepara y se activa con un comando.

Comandos de gestión

ACTIVAR Y DESACTIVAR

```
a2ensite sitio1      # virtual host
a2dissite sitio1
a2enmod  rewrite     # módulo
a2dismod rewrite
a2enconf security    # fragmento
a2disconf security
```

COMPROBAR ANTES DE RECARGAR

```
apache2ctl configtest # sintaxis
apache2ctl -S          # virtual hosts
apache2ctl -M          # módulos cargados
systemctl reload apache2
```

`apache2ctl -S` muestra qué virtual hosts hay y **cuál es el de por defecto**; `apache2ctl -M`, si un módulo (`rewrite`, `proxy` ...) está cargado.

Ejercicio rápido 2: Explorar la configuración

≈ 5 MINUTOS

1. `ls -l /etc/apache2/sites-enabled/` : ¿qué tipo de fichero hay y a dónde apunta?
2. `apache2ctl -S` : ¿cuál es el virtual host por defecto?
3. `apache2ctl -M | grep rewrite` : ¿está cargado? Actívalo y vuelve a comprobarlo.
4. Abre `ports.conf` : ¿en qué puerto escucha?

Virtual hosts en Apache

Cada sitio se define en un fichero de `sites-available/`:

```
<VirtualHost *:80>
    ServerName  www.ejemplo.org
    ServerAlias  ejemplo.org
    DocumentRoot /var/www/ejemplo
    ErrorLog     ${APACHE_LOG_DIR}/ejemplo_error.log
    CustomLog    ${APACHE_LOG_DIR}/ejemplo_access.log combined
</VirtualHost>
```

- `ServerName` / `ServerAlias` : nombres del sitio · `DocumentRoot` : directorio raíz
- Logs propios en `${APACHE_LOG_DIR}` (`/var/log/apache2`), con el formato `combined`
- Se activa con `a2ensite ejemplo` y `systemctl reload apache2`

¿Qué virtual host responde?

Apache compara la cabecera `Host` con el `ServerName` y los `ServerAlias` de cada virtual host.

- Si coincide con alguno, responde ese sitio
- Si **no coincide con ninguno**, responde el **primero que se ha cargado**
- Los ficheros se cargan por **orden alfabético**: por eso el sitio por defecto de Debian se llama `000-default.conf`

 Si desactivas `000-default`, el sitio por defecto pasa a ser el primero de los tuyos por orden alfabético.

Ejercicio rápido 3: Tu primer virtual host

≈ 5 MINUTOS

1. Crea `www.ejemplo.org` con `DocumentRoot /var/www/ejemplo` y un `index.html` propio. Actívalo.
2. En tu equipo, añade el nombre a `/etc/hosts` con la IP del servidor.
3. `curl http://www.ejemplo.org` y `curl http://<ip de acceso>`: ¿qué sitio responde en cada caso?
4. Desactiva `000-default` y repite: ¿qué ha cambiado y por qué?

Cambiar el puerto

Útil para tener **varios servicios en una máquina** (por ejemplo, Apache y Nginx a la vez).

APACHE

```
# /etc/apache2/ports.conf  
Listen 8080
```

```
<VirtualHost *:8080>  
    ...  
</VirtualHost>
```

NGINX

```
server {  
    listen 8080;  
    ...  
}
```

Para probarlo, el puerto va en la URL: `curl http://www.ejemplo.org:8080`

Directivas en bloques <Directory>

Configuran **opciones por directorio** del sistema de ficheros.

```
<Directory /var/www/ejemplo>
  Options Indexes FollowSymLinks
  AllowOverride None
  Require all granted
</Directory>
```

- **Options** — **Indexes** lista el directorio si no hay `index.html`; **FollowSymLinks** sigue enlaces simbólicos
- **AllowOverride** — qué se puede cambiar con ficheros `.htaccess` (los vemos más adelante)
- **Require** — quién puede acceder

Qué directorios puede servir Apache

/ETC/APACHE2/APACHE2.CONF

```
<Directory />
    Require all denied
</Directory>

<Directory /var/www/>
    Options Indexes FollowSymLinks
    Require all granted
</Directory>

#<Directory /srv/>
#     Options Indexes FollowSymLinks
#     Require all granted
#</Directory>
```

QUÉ SIGNIFICA

- Se **deniega todo** el disco y solo se permite `/var/www` (y `/usr/share`)
- Un `DocumentRoot` o un alias **fuera** de ellos da **403**: necesita su `<Directory>` con `Require all granted`
- Para usar `/srv` como base de los sitios, basta con **descomentar** su bloque
- En `/var/www` hay `Indexes`: sin índice, **se lista** el directorio
- **Nginx** no tiene esta restricción: solo cuentan los permisos de `www-data`

Alias y redirecciones

ALIAS

```
Alias "/imagenes" "/srv/recursos/img"  
  
<Directory "/srv/recursos/img">  
    Require all granted  
</Directory>
```

El directorio real necesita su bloque
`<Directory>` con permiso de acceso.

REDIRECT

```
Redirect "/antigua" "/nueva"  
  
Redirect permanent "/uno" \  
    "http://www.pagina2.com/dos"
```

`permanent` envía un **301**; sin él, un **302**.

Redirigir solo la raíz

`Redirect` compara por **prefijo**: `Redirect "/"` afecta a **todas** las URL, también a la de destino.

```
# MAL: bucle, /principal también empieza por /  
Redirect "/" "/principal/"  
  
# BIEN: solo la raíz, con una expresión regular  
RedirectMatch "^/$" "/principal/"
```

 `RedirectMatch` usa expresiones regulares: `^` es el principio y `$` el final de la ruta.

Autenticación básica

```
<Directory "/var/www/ejemplo/privado">  
    AuthType Basic  
    AuthName "Zona restringida"  
    AuthUserFile "/etc/apache2/.htpasswd"  
    Require valid-user  
</Directory>
```

CREAR EL ARCHIVO DE CONTRASEÑAS

```
sudo htpasswd -c /etc/apache2/.htpasswd usuario1 # -c crea el fichero  
sudo htpasswd /etc/apache2/.htpasswd usuario2 # sin -c: añade
```

Control de acceso con **Require**

```
<Directory "/var/www/ejemplo/admin">  
    Require ip 192.168.1.0/24  
</Directory>
```

DIRECTIVA	SIGNIFICADO
<code>Require all granted</code> / <code>all denied</code>	Acceso libre / bloquear todo
<code>Require ip <red></code>	Solo desde esa red o IP
<code>Require not ip <red></code>	Todos menos esa red (dentro de <code>RequireAll</code>)
<code>Require valid-user</code>	Cualquier usuario autenticado

Combinar condiciones

<REQUIREANY> : BASTA CON UNA

```
<RequireAny>
  Require ip 192.168.1.0/24
  Require valid-user
</RequireAny>
```

Desde la red indicada entra **sin contraseña**;
desde fuera **se pide**.

<REQUIREALL> : TODAS

```
<RequireAll>
  Require ip 192.168.1.0/24
  Require valid-user
</RequireAll>
```

Solo desde esa red **y** con usuario autenticado.

 Varios `Require` sueltos equivalen a `<RequireAny>`. Las directivas `AuthType`, `AuthName` y `AuthUserFile` van fuera del bloque.

Ficheros `.htaccess`

Permiten cambiar la configuración **de un directorio** sin tocar el virtual host ni recargar el servidor.

- Se colocan **en el propio directorio** del contenido
- Solo funcionan si `AllowOverride` lo permite en su `<Directory>`

ALLOWOVERRIDE	PERMITE EN <code>.HTACCESS</code>
None	Nada: el fichero se ignora (por defecto)
AuthConfig	Autenticación
FileInfo	Redirecciones y <code>mod_rewrite</code>
All	Todo

Reescritura con `mod_rewrite`

Se activa con `a2enmod rewrite` y reiniciando Apache.

```
RewriteEngine On
# /producto/5 → producto.php?id=5 (la URL no cambia)
RewriteRule "^producto/([0-9]+)$" "producto.php?id=$1" [L]
```

- Expresiones regulares: `^` principio · `$` final · `[0-9]+` uno o más dígitos · `.*` cualquier cosa · `( )` guarda lo que coincide en `$1`, `$2` ...
- `[L]`: última regla, no se aplican las siguientes
- En un `.htaccess`, la ruta **no empieza por** `/`; en el virtual host, sí

Redirecciones con `mod_rewrite`

Con la opción `R`, la regla deja de ser una reescritura y pasa a ser una **redirección**:

```
RewriteEngine On
# /antigua/... → /nueva/... (el cliente recibe un 301)
RewriteRule "^antigua/(.*)$" "/nueva/$1" [R=301,L]
```

- `[R=301]` redirección permanente · `[R=302]` temporal
- Para redirecciones sencillas basta con `Redirect` / `RedirectMatch`; `mod_rewrite` se usa cuando hacen falta **patrones o condiciones**

PHP en Apache

```
sudo apt install libapache2-mod-php
```

- Instala PHP y el módulo `mod_php`, que se activa solo
- `mod_php` no admite hilos: Debian cambia el MPM a **prefork** automáticamente
- Los ficheros `.php` del sitio se ejecutan y se devuelve el HTML generado

```
<?php  
echo "<h1>Servidor: " . gethostname() . "</h1>";  
?>
```

 Alternativa: **PHP-FPM** (`php-fpm`), un servidor de aplicaciones aparte. Es lo que se usa con Nginx.

Módulos y seguridad básica

MÓDULO	FUNCIÓN
<code>mod_rewrite</code>	Reescritura de URLs
<code>mod_proxy</code>	Proxy inverso y balanceo de carga
<code>mod_headers</code>	Manipulación de cabeceras HTTP

OCULTAR LA VERSIÓN DEL SERVIDOR

En `/etc/apache2/conf-available/security.conf`:

```
ServerTokens Prod      # cabecera Server: Apache (sin versión)
ServerSignature Off     # sin versión en las páginas de error
```

Logs en Apache

Por defecto en `/var/log/apache2/` (`access.log` , `error.log`), o en los ficheros que indique cada virtual host.

```
LogFormat "%h %l %u %t \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\"" combined
```

CAMPO	CONTENIDO
<code>%h</code>	IP del cliente
<code>%u</code>	Usuario autenticado
<code>%r</code> · <code>%>s</code>	Línea de petición · código de estado
<code>%{Cabecera}i</code>	Valor de una cabecera de la petición

Ejercicio rápido 5: Leer los logs

≈ 5 MINUTOS

1. Deja abierto `tail -f` sobre el log de acceso de `www.ejemplo.org`.
2. Desde tu equipo, repite peticiones de los ejercicios anteriores: localiza en el log la IP, la URL y el código de cada una.
3. Haz una petición con `curl -A "soy-yo"` y búscala en el log.
4. Quita a `www-data` el permiso de lectura de un fichero (`chmod 600`) y pídelo: ¿qué código obtienes? ¿Qué dice el `error.log` ?

03

Nginx

Instalación, configuración y funcionalidades

Nginx

- Servidor **web y proxy inverso** ligero, de **alto rendimiento**
- **Software libre** (BSD simplificada); existe versión comercial **Nginx Plus**
- Creado por **Igor Sysoev** en 2004 para el portal ruso *Rambler*
- Modelo **asíncrono orientado a eventos**: pocos procesos (*workers*) y cada uno atiende **miles de conexiones** a la vez, sin un proceso o hilo por conexión
- Usa **menos memoria** que Apache y es más rápido, sobre todo con **contenido estático**
- **Menos flexible** en configuraciones dinámicas (no hay `.htaccess`)

Instalación de Nginx

```
sudo apt update  
sudo apt install nginx
```

- El servicio queda activado (`nginx.service`), en el puerto **80**
- Sirve por defecto desde `/var/www/html/`

```
sudo nginx -t                # comprobar la sintaxis antes de recargar  
sudo systemctl reload nginx
```

 Nginx y Apache no pueden escuchar en el mismo puerto a la vez: en la misma máquina hay que cambiar el puerto de uno de los dos.

Ejercicio rápido 6: Instalar Nginx

≈ 5 MINUTOS

1. Con Apache en marcha, instala `nginx`. ¿Arranca? Busca el motivo con `systemctl status nginx`.
2. Para y deshabilita Apache, y arranca Nginx.
3. `curl -I` desde tu equipo: ¿qué cabecera `Server` ves ahora?
4. `ps aux | grep nginx`: ¿qué usuario tienen el proceso principal y los *workers*?

Estructura de `/etc/nginx/`

RUTA	PARA QUÉ SIRVE
<code>nginx.conf</code>	Archivo principal de configuración
<code>conf.d/</code>	Fragmentos de configuración cargados por defecto
<code>sites-available/</code> · <code>sites-enabled/</code>	Virtual hosts disponibles · activos
<code>snippets/</code>	Fragmentos reutilizables (con <code>include</code>)
<code>/var/log/nginx/</code>	Logs de acceso y errores

i Mismo patrón `sites-available/` + `sites-enabled/` que Apache, pero **sin** comandos tipo `a2ensite` : el enlace simbólico se crea a mano.

Ejercicio rápido 7: Explorar Nginx

≈ 5 MINUTOS

1. `ls -l /etc/nginx/sites-enabled/` : ¿qué sitio hay activo y a dónde apunta?
2. `sudo nginx -T | less` : muestra **toda** la configuración que se carga. Busca el `server` con `default_server`.
3. En `nginx.conf`, ¿con qué usuario se ejecutan los *workers* (`user`)? ¿Cuántos hay (`worker_processes`)? Compáralo con el número de CPU (`nproc`) y con `ps aux | grep nginx`.

Server blocks (virtual hosts)

Cada sitio se define en un fichero de `sites-available/`:

```
server {
    listen 80;
    server_name www.ejemplo.org ejemplo.org;
    root /var/www/ejemplo;
    index index.html;
    access_log /var/log/nginx/ejemplo_access.log;
    location / {
        try_files $uri $uri/ =404;
    }
}
```

- `try_files`: busca el fichero, luego el directorio y, si no existe ninguno, devuelve **404**
- Se activa con `ln -s /etc/nginx/sites-available/ejemplo /etc/nginx/sites-enabled/`, `nginx -t` y `systemctl reload nginx`

Servidor por defecto

Si el `Host` no coincide con ningún `server_name`, responde el bloque marcado con `default_server` (si no hay ninguno, el **primero** que se carga):

```
server {  
    listen 80 default_server;  
    server_name _;  
    root /var/www/html;  
}
```

 El nombre `_` es un convenio para "ningún nombre concreto"; lo que de verdad importa es `default_server` en `listen`. En Debian lo tiene el sitio `default`.

Ejercicio rápido 8: Server blocks

≈ 5 MINUTOS

1. Crea en Nginx el sitio `www.ejemplo.org` con el mismo contenido (`/var/www/ejemplo`). Actívalo.
2. Repite las pruebas del ejercicio 4 (`/docs/a.html` , `/docs` , `/docs/` , `/nada.html`): ¿cambia algún código? ¿Por qué?
3. ¿Qué sitio responde al entrar por la IP?

Bloques `location`

Asocian **URL** a una configuración. Nginx elige el bloque así:

TIPO	EJEMPLO	CUÁNDO SE USA
Exacta	<code>location = / { }</code>	Solo esa URL; tiene prioridad
Prefijo	<code>location /docs/ { }</code>	Se elige el prefijo más largo
Prefijo preferente	<code>location ^~ /img/ { }</code>	Como prefijo, pero no mira las regex
Expresión regular	<code>location ~ /\.php\$ { }</code>	La primera que coincide (<code>~*</code> sin distinguir mayúsculas)

 Orden: exacta → prefijo más largo (si es `^~`, se queda) → regex en orden → prefijo más largo.

Ejercicio rápido 9: ¿Qué `location` responde?

≈ 10 MINUTOS

En el sitio `www.ejemplo.org`, sustituye la `location /` por estos bloques (`return 200 "texto"` responde directamente con ese texto):

```
location = /      { return 200 "exacta /\n"; }  
location /        { return 200 "prefijo /\n"; }  
location /docs/   { return 200 "prefijo /docs/\n"; }  
location ^~ /img/ { return 200 "preferente /img/\n"; }  
location ~ /\.png$ { return 200 "regex .png\n"; }
```

Antes de probarlo, apunta qué bloque crees que responde a `/`, `/index.html`, `/docs/a.html`, `/docs/a.png` e `/img/a.png`. Después compruébalo con `curl`.

root y alias

ROOT : AÑADE LA URL COMPLETA

```
location /img/ {  
    root /srv;  
}
```

/img/logo.png → /srv/img/logo.png

ALIAS : SUSTITUYE EL PREFIJO

```
location /imagenes/ {  
    alias /srv/recursos/img/;  
    autoindex on;  
}
```

/imagenes/logo.png →

/srv/recursos/img/logo.png

- **autoindex on** muestra el listado del directorio si no hay **index**
- Con **alias**, pon la **barra final** en la **location** y en la ruta

Autenticación básica

```
location /privado/ {  
    auth_basic          "Zona restringida";  
    auth_basic_user_file /etc/nginx/.htpasswd;  
}
```

El fichero de contraseñas se crea con `htpasswd` (paquete `apache2-utils`):

```
sudo apt install apache2-utils  
sudo htpasswd -c /etc/nginx/.htpasswd usuario1  
sudo htpasswd    /etc/nginx/.htpasswd usuario2
```

 El formato del fichero es el mismo que en Apache.

Control de acceso por IP

```
location /admin/ {  
    allow 192.168.1.0/24;  
    allow 10.0.0.5;  
    deny all;  
}
```

- Las reglas se comprueban **en orden**: se aplica la **primera** que coincide
- Si se deniega, responde **403**

Combinar IP y autenticación

```
location /panel/ {  
    satisfy any;          # basta con cumplir UNA de las dos  
  
    allow 192.168.1.0/24;  
    deny all;  
  
    auth_basic            "Acceso restringido";  
    auth_basic_user_file /etc/nginx/.htpasswd;  
}
```

- **satisfy any**: desde la red indicada entra **sin contraseña**; desde fuera **se pide**
- **satisfy all** (por defecto): exige las **dos** condiciones

Redirecciones con `return`

```
# Toda la web, permanente (301)
return 301 http://nuevo.ejemplo.org$request_uri;

# Solo la raíz: location exacta
location = / {
    return 302 /principal/;
}
```

- `return` es la forma **recomendada** de redirigir
- `$request_uri` conserva la ruta y la consulta originales
- Con `location = /` solo se redirige la raíz: el resto de URL no entra en ese bloque

Reescritura con `rewrite`

```
# Reescritura: /producto/5 → /producto.php?id=5 (la URL no cambia)
rewrite ^/producto/([0-9]+)$ /producto.php?id=$1 last;

# Redirección: el cliente recibe un 301
rewrite ^/noticias/(.*)$ /novedades/$1 permanent;
```

OPCIÓN	EFFECTO
<code>last</code>	Reescritura interna; vuelve a buscar la <code>location</code>
<code>break</code>	Reescritura interna dentro de la misma <code>location</code>
<code>redirect</code> · <code>permanent</code>	Redirección 302 · 301

Snippets y seguridad básica

Fragmentos en `/etc/nginx/snippets/` que se incluyen desde varios `server`:

```
# /etc/nginx/snippets/privado.conf
auth_basic          "Zona restringida";
auth_basic_user_file /etc/nginx/.htpasswd;
```

```
location /admin/    { include snippets/privado.conf; }
location /informes/ { include snippets/privado.conf; }
```

 Para ocultar la versión en la cabecera `Server`: `server_tokens off;` en `nginx.conf`.

PHP con PHP-FPM

Nginx no ejecuta PHP: pasa las peticiones de ficheros `.php` a **PHP-FPM**, un servidor de aplicaciones, con el protocolo **FastCGI**. Se comunican por un **socket**: un fichero especial del sistema.

```
sudo apt install php-fpm
```

```
index index.php index.html;

location ~ \.php$ {
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php8.4-fpm.sock;
}
```

 El nombre del socket depende de la versión de PHP: compruébalo con `ls /run/php/`.

Logs en Nginx

Por defecto en `/var/log/nginx/` (`access.log`, `error.log`), o en los que indique cada `server` .

- `access.log` usa el formato `combined` , el mismo que Apache
- `error.log` recoge errores de configuración, permisos y backends (PHP-FPM, `proxy_pass` ...)

```
log_format personalizado '$remote_addr "$request" $status "$http_user_agent";  
access_log /var/log/nginx/ejemplo_access.log personalizado;
```

`$http_<cabecera>` es el valor de una cabecera de la petición (en minúsculas y con `_`).

Ejercicio rápido 10: Apache y Nginx a la vez

≈ 5 MINUTOS

1. Cambia Nginx para que escuche en el puerto **8080** y vuelve a arrancar Apache en el 80.
2. Comprueba con `sudo ss -tlnp` qué proceso escucha en cada puerto.
3. Desde tu equipo, pide `http://www.ejemplo.org` y `http://www.ejemplo.org:8080`: ¿qué cabecera `Server` devuelve cada uno?
4. ¿En qué log aparece cada petición?

04

Resumen

Equivalencias y problemas frecuentes

Equivalencias: configuración

	APACHE	NGINX
Sitio	<code><VirtualHost *:80></code>	<code>server { listen 80; }</code>
Nombre	<code>ServerName</code> / <code>ServerAlias</code>	<code>server_name</code>
Directorio raíz	<code>DocumentRoot</code>	<code>root</code>
Puerto	<code>Listen</code> en <code>ports.conf</code> + <code><VirtualHost *:8080></code>	<code>listen 8080</code>
Sitio por defecto	El primero cargado (<code>000-default</code>)	<code>listen 80 default_server</code>
Activar sitio	<code>a2ensite</code>	<code>ln -s a sites-enabled/</code>
Comprobar	<code>apache2ctl configtest</code>	<code>nginx -t</code>
Por directorio	<code>.htaccess</code>	—

Equivalencias: funcionalidades

	APACHE	NGINX
Alias	Alias + <Directory>	location + alias
Listado	Options Indexes	autoindex on
Redirección	Redirect / RedirectMatch	return 301
Reescritura	RewriteRule ... [L]	rewrite ... last
Acceso por IP	Require ip	allow / deny
Autenticación	AuthType Basic + Require valid-user	auth_basic
IP o usuario	<RequireAny>	satisfy any
PHP	mod_php	PHP-FPM

Problemas frecuentes

SÍNTOMA	CAUSAS HABITUALES
403	Permisos de <code>www-data</code> · control de acceso · sin índice ni listado
404	<code>DocumentRoot</code> , <code>root</code> o <code>alias</code> mal · el fichero no está
Sale otro sitio	El nombre no resuelve · no coincide con <code>ServerName</code> / <code>server_name</code> · sitio sin activar o sin recargar
500	Error de configuración (<code>.htaccess</code> , módulo sin activar) o de la aplicación
502	Nginx no llega a PHP-FPM (socket mal o servicio parado)
No arranca	Error de sintaxis · puerto ocupado

Antes de recargar: `apache2ctl configtest` o `nginx -t` . Ante un error: el `error.log` .

Para profundizar

- [Documentación oficial de Apache](#)
- [Documentación oficial de Nginx](#)

SRI · UNIDAD 2 — SERVIDORES WEB: APACHE Y NGINX

¡Gracias!

Servidores web

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI